

EE-170L Computer Programming Lab

Laboratory Report — No. 07

One-Dimensional Array in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	7
Topic	1D Arrays, Character Arrays
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand arrays as fixed-size, same-type sequential collections in C++.
- Declare and initialize one-dimensional integer and character arrays.
- Access and modify individual elements using zero-based index notation.
- Iterate over array elements using for loops for display and computation.
- Copy array contents element-by-element between two arrays using a loop.
- Apply array operations to solve practical input, output, and calculation tasks.

2. Theoretical Background

2.1 What is an Array?

An array is a data structure in C++ that stores a fixed-size sequential collection of elements of the same data type. All array elements occupy contiguous memory locations — the lowest address holds the first element and the highest address holds the last.

Arrays are useful when multiple values of the same type must be stored and processed together, such as a list of marks, temperatures, or ages.

2.2 Declaration of Array

To declare a 1D array, specify the data type, array name, and size:

Syntax:

```
dataType arrayName[arraySize];
```

```
// Example:
```

```
int age[4];
```

- `int` — data type of each element.
- `age` — name of the array.
- `[4]` — size; the array holds exactly 4 integers.

2.3 Initialization of Array

Arrays can be initialized at declaration by providing values inside curly braces:

```
int age[4] = {23, 34, 64, 75};
```

Individual elements can also be assigned after declaration using their index:

```
age[0] = 23;
age[1] = 34;
age[2] = 64;
age[3] = 75;
```

2.4 Accessing Elements of Array

Array elements are accessed using `arrayName[index]`. Indices are zero-based, so the first element is `arrayName[0]` and the last is `arrayName[size-1]`.

Syntax:

```
arrayName[index];

age[0]; // 1st element
age[1]; // 2nd element
age[2] = 75; // assign to 3rd element
int a = age[2]; // read 3rd element
```

2.5 Copying Arrays

C++ has no single statement to copy an entire array. Elements must be copied one by one using a loop:

```
#include <iostream>
using namespace std;
int main()
{
    int age[4] = {23, 34, 64, 75};
    int same_age[4];
    for (int i = 0; i < 4; i++)
    {
        same_age[i] = age[i];
    }
    for (int i = 0; i < 4; i++)
        cout << same_age[i] << "\t";
    return 0;
}
```

3. Example Programs

3.1 Declare, Initialize, and Display an Integer Array

```
#include <iostream>
using namespace std;
int main()
{
    int age[4];
    age[0] = 23; age[1] = 34; age[2] = 64; age[3] = 75;
    for (int i = 0; i < 4; i++)
        cout << age[i] << "\t";
    return 0;
}
```

Output:

```
23    34    64    75
```

3.2 Copy Array Using a for Loop

The following example declares two arrays, fills the first with values, then copies each element into the second using a for loop:

```
#include <iostream>
using namespace std;
int main()
{
    int source[4] = {10, 20, 30, 40};
    int dest[4];
    for (int i = 0; i < 4; i++) dest[i] = source[i];
    for (int i = 0; i < 4; i++)
        cout << dest[i] << "\t";
    return 0;
}
```

Output:

```
10    20    30    40
```

4. Lab Tasks & Solutions

Task 1 — Array Declaration and Initialization

Declare an integer array 'numbers' of size 5, initialize it with {10, 20, 30, 40, 50}, and display all elements using a loop.

```
#include <iostream>
using namespace std;
int main()
{
    int numbers[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        cout << "numbers[" << i << "] = " << numbers[i] << endl;
    return 0;
}
```

How it works:

- Array is declared and initialized in a single statement using curly-brace syntax.
- The for loop iterates from index 0 to 4, printing both the index and value.
- Total output: 5 lines, one per element.

Task 2 — Accessing Array Elements — Character Array

Declare a character array 'message' storing the word 'HELLO'. Use a loop to display each character on a separate line.

```
#include <iostream>
using namespace std;
int main()
{
    char message[5] = {'H', 'E', 'L', 'L', 'O'};
    for (int i = 0; i < 5; i++)
        cout << message[i] << endl;
    return 0;
}
```

How it works:

- A char array stores individual characters, each accessible by its index.
- The loop accesses message[0] through message[4] in sequence.
- Each character is printed on its own line using endl.

Task 3 — Copying Arrays

Declare 'source[5]' and 'destination[5]'. Initialize source, copy element-by-element, then display both arrays.

```
#include <iostream>
using namespace std;
int main()
{
    int source[5]      = {5, 10, 15, 20, 25};
    int destination[5];
    for (int i = 0; i < 5; i++) destination[i] = source[i];
    cout << "Source:      ";
    for (int i = 0; i < 5; i++) cout << source[i]      << " ";
    cout << endl;
    cout << "Destination: ";
    for (int i = 0; i < 5; i++) cout << destination[i] << " ";
    cout << endl;
    return 0;
}
```

Output:

```
Source:      5 10 15 20 25
Destination: 5 10 15 20 25
```

Task 4 — Array Sum

Prompt the user to enter 5 integers, store in an array, compute and display the total sum.

```
#include <iostream>
using namespace std;
int main()
{
    int nums[5], sum = 0;
    cout << "Enter 5 integers:" << endl;
    for (int i = 0; i < 5; i++)
    {
        cout << "Number " << i+1 << ": ";
        cin >> nums[i];
    }
    for (int i = 0; i < 5; i++) sum += nums[i];
    cout << "Sum = " << sum << endl;
    return 0;
}
```

How it works:

- First loop collects 5 user-entered integers into the array.
- Second loop accumulates all elements into sum using `sum += nums[i]`.
- Final sum is displayed after both loops complete.

5. Array vs Regular Variable — Key Differences

Feature	Regular Variable	Array
Stores	One value	Multiple same-type values
Declaration	<code>int x;</code>	<code>int x[10];</code>
Access	<code>x</code>	<code>x[0], x[1], ... x[n-1]</code>
Memory	Single location	Contiguous block
Loop use	Not needed	Essential for traversal

Size	Fixed at 1	Fixed at declaration (n values)
------	------------	---------------------------------

6. Conclusion

This lab introduced one-dimensional arrays in C++ — a fundamental data structure for handling collections of same-type data. Key takeaways:

- **Arrays:** Store multiple values of the same type in a single named variable using contiguous memory.
- **Index-Based Access:** Elements accessed via zero-based indices — `arrayName[0]` to `arrayName[size-1]`.
- **Initialization:** Arrays can be initialized at declaration with curly braces or assigned element-by-element.
- **Loop Integration:** for loops are essential for traversing, copying, and computing on array elements.
- **Character Arrays:** Char arrays store strings character-by-character and follow the same access rules.

Mastery of 1D arrays is the foundation for 2D arrays, functions, and string processing covered in the upcoming labs.